



ISEN PROGRAM – MASTER 1 PROJECT

CROPSENSE

Imaging platform for plant analysis

Phase 2

Team Members:

Delia Fragă

Maria-Alexandra Molnar

Olesia Yankiv

Antoine Herbaux

Chiemerie David Ekweanua

Supervisors:

Herinaina Andriamandroso

Iryna Kyrylyuk

Lille, France

April 24, 2026

Table of Contents

1. Introduction.....	4
2. System Components.....	4
2.1. Imaging Hardware and Acquisition Protocol.....	4
2.1.1. Hyperspectral Camera	4
2.1.2. Camera Protocol.....	5
2.1.2.1 Camera Attributes	5
2.1.2.2. Acquisition Order Rationale	5
2.1.2.3. Hyperspectral Camera Protocol	5
2.1.2.4. Multispectral Camera Protocol.....	6
2.1.2.5. Thermal Camera Protocol	6
2.1.3. Quality Checks	6
2.1.4. Limitations and improvement suggestions	7
2.2. Data Management and Database	7
2.2.1 Database Architecture	8
2.3. Data Processing Pipeline	10
2.3.1. Automated Data Ingestion and Sorting.....	10
2.3.2. Data Transformation and Sensor Fusion	11
2.3.3. Data Extraction, Visualization, and Agronomic Insights	12
2.4. Web Application.....	15
2.4.1. Introduction	15
2.4.2. Technologies used	15
2.4.3. Architecture of the application.....	16
2.4.3.1. Presentation Layer (Frontend)	16
2.4.3.2. Application Layer (Controllers)	17
2.4.3.3. Service Layer (Business Logic).....	17
2.4.3.4. Data Storage Layer	17
2.4.3.5. Deployment Layer	17

2.4.4. Application Pages	17
3. System Integration	19
4. Project Management	21
4.1 Group 1: Website Development.....	21
4.2 Group 2: Database Implementation and ETL Pipeline Development	21
4.3 Group 3: Cameras Usage Protocol.....	22
5. Future Perspectives.....	22
6. Conclusion.....	28
7. References	29

1. Introduction

Multispectral, hyperspectral, and thermal imaging have been widely demonstrated in the scientific literature as effective tools for detecting plant stress and disease, particularly through the identification of subtle physiological changes that occur prior to the appearance of visible symptoms (Mahlein, 2016). Within this context, the CropSense project forms part of a broader research initiative conducted as part of the doctoral work of Iryna Kyrylyuk (Trans-e-bio Project WP5 part of the Biocontrol 4.0 portfolio), which focuses on the early detection of wheat diseases, such as *Zymoseptoria tritici*, using advanced imaging techniques and data analysis methods. The objective of this research is to develop reliable and non-invasive approaches for monitoring plant health, thereby contributing to the advancement of precision agriculture.

Phase 2 of the CropSense project aims to simplify data acquisition, management or storage and processing by developing an intuitive user interface that enables users to easily visualize and explore collected data. This approach is intended to support the future development of predictive models capable of accurately identifying plant diseases.

The following sections describe each component required for the system, followed by a section detailing their integration, and concluding with future perspectives.

2. System Components

2.1. Imaging Hardware and Acquisition Protocol

This section presents the imaging hardware used in this study along with their corresponding acquisition protocols. It outlines the operational principles and key parameters required to ensure consistent and reproducible data acquisition.

2.1.1. Hyperspectral Camera

The **Specim FX10** is a pushbroom (line-scan) hyperspectral camera designed to capture spectral information across the **400–1000 nm** range [1]. Unlike conventional cameras, it acquires one spatial line at a time, requiring controlled motion to reconstruct a full image [1],[2]. Consequently, image acquisition is defined as a scanning process rather than a single capture event.

The system produces hyperspectral data cubes, where each pixel contains a full spectral signature. This enables advanced analysis such as plant stress detection and phenotyping.



Figure 1 Specim Fx10 (Hyper spectral camera)

2.1.2. Camera Protocol

This section presents a structured overview of the acquisition protocols for the **hyperspectral, multispectral, and thermal cameras** used in this study. It highlights their key parameters and provides standardized procedures to ensure consistent and reproducible data acquisition.

2.1.2.1 Camera Attributes

The table below summarizes the main operational characteristics and requirements of the three imaging systems.

Parameter	Hyperspectral (Specim FX10)	Multispectral (Silos Toucan)	Thermal (WIRIS Agro)
Acquisition type	Line-scan (pushbroom)	Multi-band capture	Single image
Software	Lumo Scanner and specim insights	ColorShades Lab	No software
Mounting	Fixed vertical frame	Adjustable vertical rail	Fixed vertical pole
Calibration	White & dark reference required	White & dark reference required	Internal (automatic)
Warm-up	System stabilization required	1–2 min	~2 min 10 s
Lighting sensitivity	High	High	Low
Acquisition complexity	High	Medium	Low
Estimated Time per Plant	Slow (2-4 minutes)	~10–15 s	~1 s

Table 1 Camera attributes

2.1.2.2. Acquisition Order Rationale

The order **hyperspectral** → **multispectral** → **thermal** is chosen to ensure reliable data acquisition. Hyperspectral imaging is performed first because it is the most sensitive and requires stable conditions and controlled motion. Multispectral imaging follows, as it is simpler but still depends on consistent acquisition conditions. Thermal imaging is done last since it is independent of visible light and less affected by setup changes. This order reduces interference between systems and ensures consistent measurements.

2.1.2.3. Hyperspectral Camera Protocol

Acquisition Parameters	Acquisition Steps
Scan speed: 9.0	Connect camera and motor system to PC
Positioning speed: 50.0	Launch Lumo Scanner

Frame rate: 50 fps	Set parameters (scan speed, frame rate, exposure)
Exposure time: 8 ms	Perform white reference calibration
Working distance: ~27–29 cm	Position plant under camera
Acquisition mode: Manual	Start scan ensuring smooth motion
Software: Lumo Scanner / Specim Insight	Save hyperspectral cube

Table 2 Hyperspectral camera protocol

Include additional parameters: Focus Grid (5), Target Start (44), Dark Start (5), Dark Stop (10), White Start (88), and White Stop (134).

2.1.2.4. Multispectral Camera Protocol

Acquisition Parameters	Acquisition Steps
Focus distance: ~1 m	Connect camera via USB to Pc
Aperture: f/2.8 – f/4	Launch ColorShades Lab
Integration time: ~8.33 ms	Perform white and dark calibration
Frame rate: ~60 fps (software)	Position plant under camera
Software: ColorShades Lab	Capture and Save image

Table 3 Multispectral camera protocol

2.1.2.5. Thermal Camera Protocol

Acquisition Parameters	Acquisition Steps
Warm-up time: ~2 min 10 s	Mount camera and connect to power
Power supply: 12 V battery	Allow warm-up period
Capture time: ~1 s per image	Ensure internal calibration is complete
Frame rate: ~50 Hz (video output)	Position plant at fixed distance
Calibration: Internal (NUC)	Capture and Save image

Table 4 Thermal Camera Protocol

2.1.3. Quality Checks

The following quality checks should be performed before and after image acquisition to ensure data consistency and reliability.

Pre-acquisition	Post-Acquisition
Ensure the object is centered in the field of view	Check that the image is complete (no missing parts)
Verify focus using a reference object (no distortion)	Make sure there are no distortions or motion errors (especially for hyperspectral scans)
Confirm white reference fully covers the field of view	Verify that the image is clear and the plant is visible

Maintain stable lighting conditions	Check that the brightness or temperature looks correct (no overexposed or dark images)
Ensure continuous motion for hyperspectral scanning (no artifacts)	Make sure the file is saved correctly with the right name

Table 5 Quality checks (pre&post acquisition)

2.1.4. Limitations and improvement suggestions

The following list elaborates limitations faced during the project and offers areas for improvement.

- The vertical range of the hyperspectral camera is currently limited, this causes taller plants to be affected when going through the back-and-forth motion of the scanner and the height needs to be increased to accommodate bigger or taller plants thereby improving the lifespan of the plants
- The current halogen lightning of the hyperspectral camera is not ideal, and I suggest an additional light source
- The monitor currently used for the thermal camera is not ideal due to its size and limited portability. A more compact and compatible solution is required; therefore, the Atomos Shinobi II is recommended. It offers high brightness for outdoor visibility, battery-powered operation for field use, and is compatible with the 50 Hz output of the thermal camera, ensuring smooth real-time monitoring.

2.2. Data Management and Database

CropSense generates heterogeneous, high-volume data from three sensor modalities — multispectral (SILIOS TOUCAN), thermal (Workswell WIRIS AGRO R), and hyperspectral (SPECIM FX10) — each producing hundreds of megabytes of raw binary files per acquisition session. It is essential to have a database that could handle both structured scientific metadata and large binary objects in a single system.

MongoDB was selected over relational alternatives, like MySQL or PostgreSQL, for three concrete reasons:

(1) Its document-oriented, schema-less model allows a capture event to record whichever sensors were active — if only a thermal camera was used, the multispectral field is simply null rather than requiring workarounds in a rigid schema.

(2) MongoDB's native GridFS subsystem provides chunked binary storage for large files without a separate file server.

(3) The pymongo driver integrates directly with the NumPy and pandas stack used throughout the pipeline, keeping the codebase unified.

2.2.1 Database Architecture

The database *cropsense_db* uses a hub-and-spoke model. A central *capture_events* collection acts as the anchor for every acquisition session, identified by a *sample_id* (plant identifier) and a *timestamp*. All sensor-specific data is stored in dedicated collections that reference the central event via a MongoDB ObjectId (*capture_event_id*).

Table 4. Collection structure of *cropsense_db*

Collection	Content	Key Fields
capture_events	One document per acquisition session	sample_id, timestamp, capture_status
sensors_multispectral	SILIOS TOUCAN metadata	capture_event_id, gridfs_ref, ndvi, histogram_256
sensors_thermal	Workswell WIRIS AGRO metadata	capture_event_id, gridfs_ref, cws_i, max_leaf_temp_c
sensors_hyperspectral	SPECIM FX10 metadata	capture_event_id, gridfs_ref, spectral_range_nm
ml_analysis	ML health classification results	capture_event_id, health_status, confidence_scores
fs.files / fs.chunks	GridFS binary storage (raw image files)	filename, length, uploadDate

2.2.2 Connection and Failover

The system connects to the database via an automatic failover strategy implemented in *get_mongo_client()*. It iterates through a priority list of URIs (remote server first, then localhost) with a 2-second timeout per attempt. This allows the pipeline to run identically in collaborative lab sessions and in offline development, without any manual configuration change.

2.2.3 10-Minute Synchronisation Window

Because the three cameras are not triggered simultaneously, files from the same session may arrive seconds or minutes apart. The *get_or_create_capture_event()* function handles this by checking whether an existing *capture_events* document already exists for the same plant within a ± 10 -minute window. If one is found, all new sensor data is linked to it. If not, a new event is created. This ensures that a complete multi-sensor session is always stored as a single logical record in the database.

2.2.4 Binary File Storage with GridFS

Raw sensor files cannot be stored directly inside MongoDB documents, which are limited to 16 MB. The system uses GridFS, MongoDB's built-in chunked binary protocol, for all raw

image data. GridFS automatically splits files into 255 KB chunks in *fs.chunks* and records metadata in *fs.files*. The returned ObjectId (*gridfs_ref*) is saved in the corresponding sensor document, linking structured metadata to its raw binary. The *save_gridfs_file()* helper performs the reverse operation — downloading a file from GridFS by ID to a local folder — used by both the export functions and the web application.

2.2.5 Data Ingestion and Management Flow

Data enters the database through *run_pipeline()*, which orchestrates the full ETL (Extract-Transform-Load) workflow. Each file follows the same four steps:

- Detection: the file is located within the structured folder hierarchy (DATA_ROOT / Date / PlantID / SensorType).
- Deduplication: the relevant collection is queried by filename to confirm the file has not already been ingested.
- Binary upload: the raw file is sent to GridFS, returning a *gridfs_ref ObjectId*.
- Metadata insertion: a structured document is built and inserted into the sensor collection, referencing the capture event.

For each ingested multispectral or thermal capture, a placeholder document is also created in *ml_analysis* with *health_status* = 'pending', signalling to the ML module that a new sample is ready for classification. An edge case is handled by *process_independent_thermal()*, which ingests thermal files that were captured without a corresponding spectral session — these are stored with *sensor_multispectral* set explicitly to null, keeping multi-sensor and standalone thermal events distinguishable in queries.

2.2.6 Data Export and Integrity

Three export utilities allow researchers to retrieve data without interacting with the database directly. *export_file_from_db()* fetches the most recent file for a given plant and sensor type. *batch_export_all()* reconstructs the entire dataset on disk from GridFS, useful for backups. *export_recent_files()* retrieves the N latest captures matching an optional plant filter. Additionally, *check_database_integrity()* verifies that every *gridfs_ref* in a sensor document points to a file that actually exists in GridFS — critical for detecting incomplete ingestions caused by network interruptions.

2.2.7 Limitations and Future Work (Data Aspect)

The current implementation addresses all core requirements but has several known limitations. No authentication or role-based access control is configured, which would be mandatory in a multi-user production environment. The 10-minute synchronisation window is a practical heuristic that could fail if two distinct sessions for the same plant occur within 20 minutes of each other — explicit session IDs assigned at acquisition time would be a more robust solution. For very large hyperspectral cubes (above 500 MB), GridFS throughput may become a bottleneck

compared to dedicated object storage systems such as MinIO. Finally, no automated backup or replica set is currently configured; implementing a MongoDB replica set with periodic mongodump snapshots would be the recommended next step to protect irreplaceable experimental data.

2.3. Data Processing Pipeline

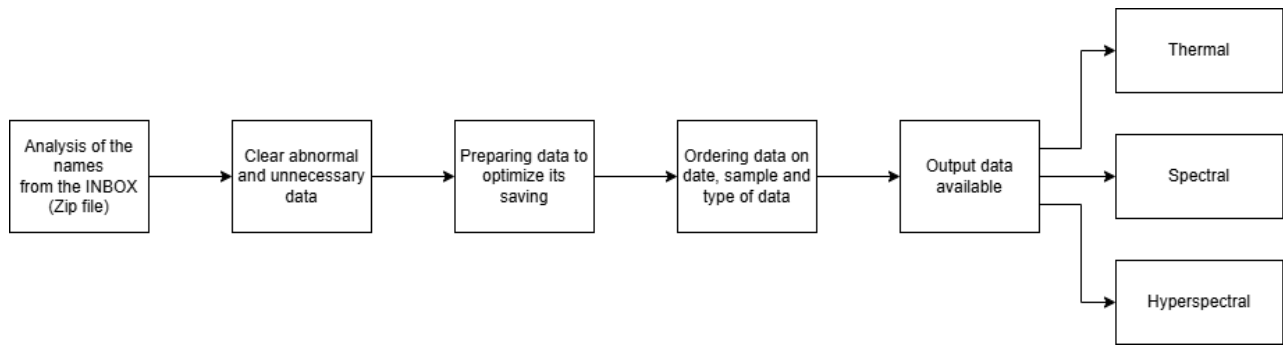
The software backbone of the CROPSENSE project relies on a comprehensive and automated Extract, Transform, and Load (ETL) pipeline. Designed to handle large volumes of complex sensor data, this pipeline acts as an intelligent orchestrator. It bridges the gap between raw hardware outputs and actionable agronomic insights, ensuring that data is seamlessly ingested, scientifically calibrated, securely stored, and easily extractable for analysis.

2.3.1. Automated Data Ingestion and Sorting

The data lifecycle is explicitly initiated by the user via the web application interface. Once an acquisition session is complete and the various camera systems have deposited their raw, unstructured files into a temporary inbox, the user triggers the "ETL Process" command. This action invokes the core `run_pipeline()` function, waking up the automated ingestion engine.

To eliminate human error and streamline operations, the pipeline immediately executes a seamless, three-step triage process:

- **Pattern Recognition & Identification:** The system utilizes advanced pattern recognition to parse the nomenclature of every file in the inbox. It automatically extracts the unique plant ID and determines the source sensor (Multispectral, Hyperspectral, or Thermal) by analyzing specific file prefixes and timestamp structures.
- **Quality Control & Data Cleansing:** Acting as a strict initial checkpoint, the pipeline performs preliminary cleansing. It autonomously detects and purges invalid files or accidental captures—such as anomalous software screenshots occasionally generated by the thermal camera interface.
- **Hierarchical Routing:** Once validated, the system dynamically transfers the raw files from the temporary inbox into a permanent local storage architecture. This repository is logically organized by date, plant identifier, and sensor type, creating a highly structured foundation for the subsequent transformation phase.



Representative schema of the Data Ingestion process

2.3.2. Data Transformation and Sensor Fusion

Raw optical data is heavily influenced by ambient lighting conditions, hardware noise, and background interference. The transformation phase is critical for converting these raw digital numbers into scientifically valid reflectance and temperature values.

Technical Reading and Calibration:

- For Multispectral data, the pipeline utilizes the `tifffile` library to open the mosaic. It then performs automatic radiometric corrections by reading auxiliary manifest files to extract exposure times and normalization coefficients.
- For the massive Hyperspectral data cubes, (which are saved line by line on 224 bands), the system uses the `spectral.io.envi` library to parse the header and load the binary data. The pipeline then applies "Dark" and "White" reference profiles captured during the experiment setup to eliminate sensor noise and calibrate the light reflection accurately.
- Thermal files are processed using a combination of `tifffile` and `OpenCV` to extract pixel-level temperature data and calculate the Crop Water Stress Index (CWSI).

A pivotal step in this transformation is the application of Semantic Masking. To ensure that background elements—such as soil, pots, or the conveyor belt itself—do not skew the statistical analysis of the plant, the algorithm generates a digital mask. By combining vegetation indices with specific brightness thresholds, the system effectively "cuts out" the plant from its background. All subsequent statistical analyses, such as calculating average reflectance, are performed exclusively on this isolated biomass.

Because the spectral and thermal cameras operate asynchronously and trigger at slightly different moments, the pipeline features a sensor fusion mechanism. The system uses the precise timestamp of a spectral image as an anchor and scans the thermal dataset to find a corresponding capture within a strict time window. When a match is found, the multimodal data is logically bound

together, creating a comprehensive, multi-dimensional snapshot of the plant at that exact moment. (Note: Once transformed and fused, data is securely pushed to the system's database, the architecture of which is detailed in Section 2.2).

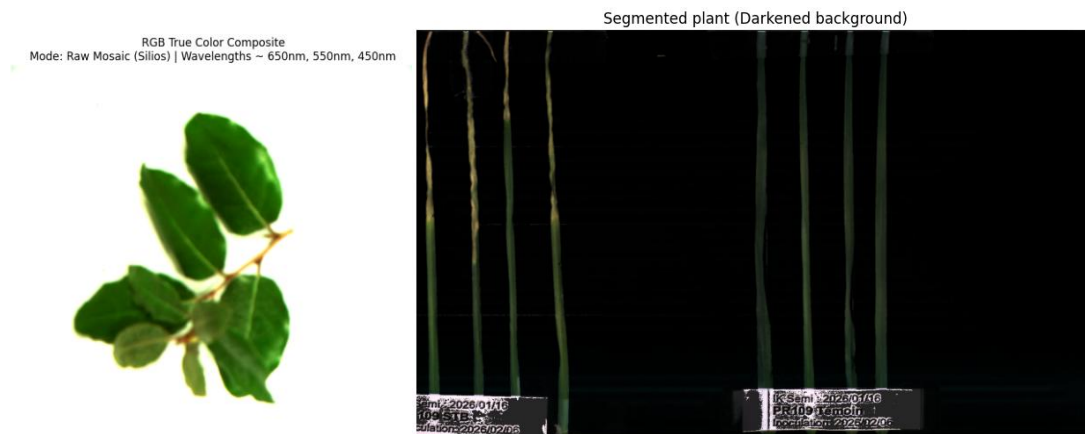
2.3.3. Data Extraction, Visualization, and Agronomic Insights

The ultimate objective of the CROPSENSE pipeline is to make invisible plant indices visible and understandable. To achieve this, the system provides powerful extraction tools (Reverse-ETL) and automated visualization capabilities.

Researchers are not required to interact directly with complex database queries or massive raw files. The system allows users to export targeted datasets directly into standard, human-readable formats utilizing the `export_analysis_to_csv()` function. Whether a researcher needs the 10-band intensity histograms of a specific plant or the spectral profile curves of an entire experimental batch, the data can be extracted instantly via `batch_export_all()` or `export_recent_files()` for use in external statistical software like Excel or R.

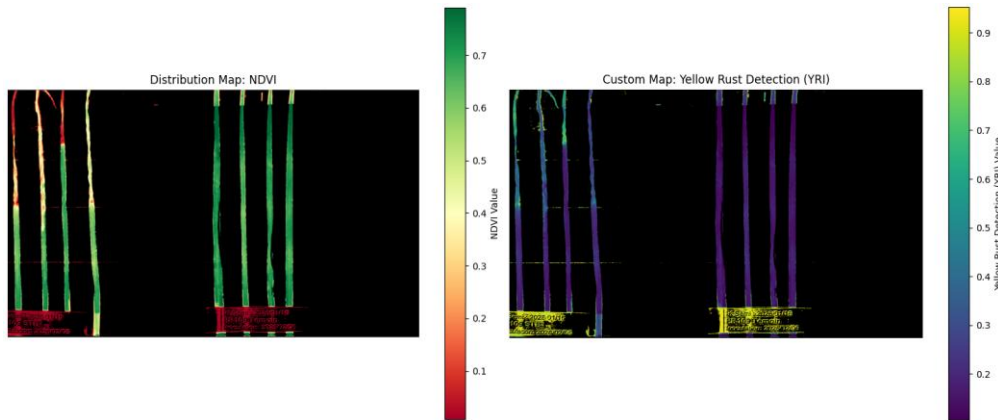
To help researchers intuitively grasp the physiological state of the plants, the pipeline automatically generates a suite of visual tools:

- **False-Color RGB Imagery:** Hyperspectral cameras capture hundreds of bands, many of which are invisible to the naked eye. Processed during the `prepare_hyperspectral_data()` sequence, the pipeline extracts specific wavelengths (for example, mapping Near-Infrared or Red-Edge bands to standard Red, Green, and Blue channels) to generate enhanced synthetic images. These false-color images highlight the plant's cellular structure, emphasize biomass, and isolate the foliage perfectly from the darkened background for immediate visual validation.



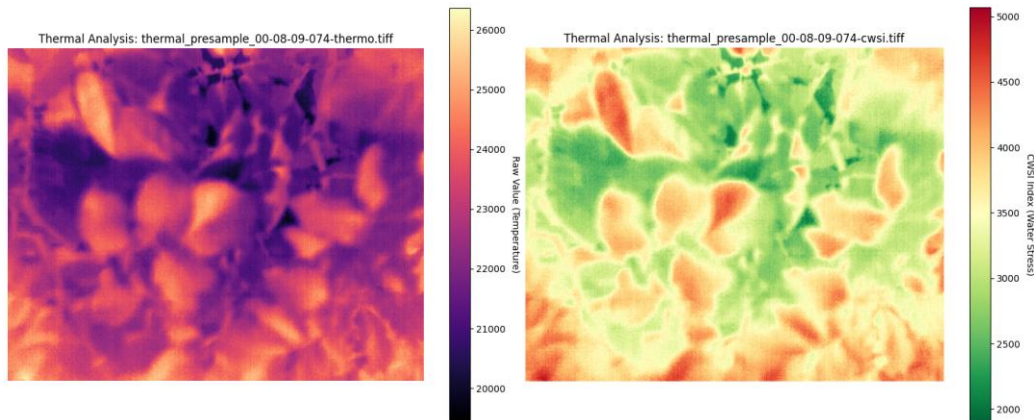
Representation in colors of various captures

- Spatial Heatmaps of Vegetation Indices:** Rather than just providing an average number, the dynamic mathematical engine maps vegetation indices directly onto the 2D spatial plane of the plant. By applying formulas like NDVI (overall vigor), PRI (photosynthetic efficiency), or custom disease metrics utilizing the `create_custom_index()` function, the system generates color-coded heatmaps displayed via `show_custom_map()`. These allow researchers to see exactly *where* the stress is localized on the leaf surface before any visible necrosis occurs.



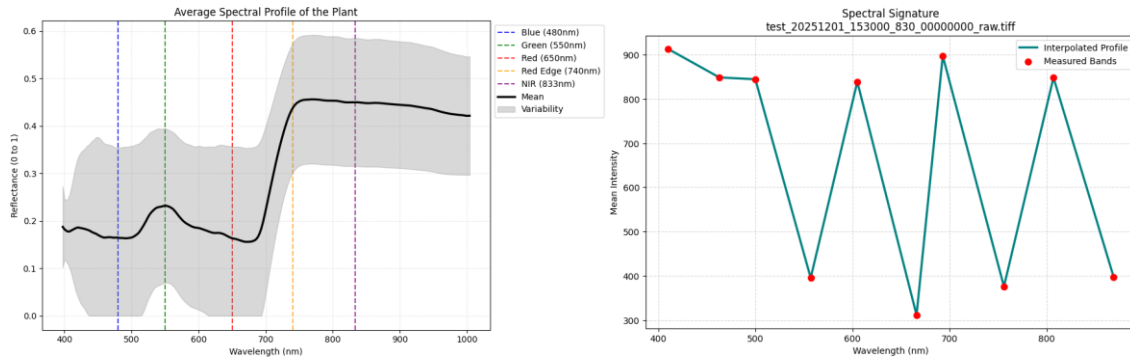
Representation of various indices on the plant (NDVI and a custom Yellow Rust Indice)

- Thermal and CWSI Heatmaps:** Correlated with the optical data, these visual maps represent temperature gradients across the plant canopy. Hotter areas, often mapped using specialized color scales, indicate stomatal closure and reduced transpiration, serving as a highly sensitive early indicator of drought stress.



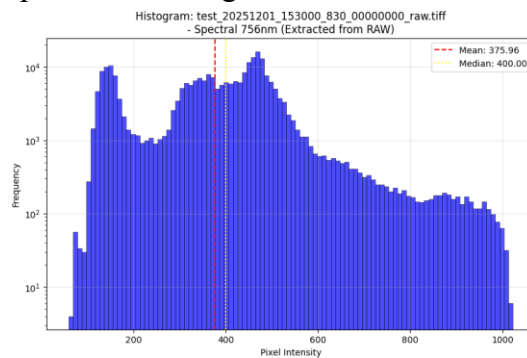
Representation of Thermal and CWSI Heatmaps

- Spectral Signature Graphs:** The system plots the mean reflectance across the entire captured light spectrum (e.g., from 400nm to 1000nm). The unique shape of this curve acts as a fingerprint for the plant's current health. Furthermore, the pipeline automatically shades the area representing the standard deviation around this curve, allowing researchers to visualize the physiological variability across the entire leaf surface.



Graphs generated from hyperspectral and spectral data

- **Multispectral Intensity Histograms:** Extracted via the `process_spectral_image()` function for the 10-band snapshot camera, the system generates distribution histograms. Instead of spatial maps, these charts visualize the frequency of pixel intensities across each specific wavelength.



Histogram generated from spectral data

- **Comparative Analytics Charts:** Through the `compare_plants_indices()` functionality, the pipeline compiles the calculated metrics of multiple plants into unified Pandas DataFrames. It can then generate comparative bar charts or scatter plots, allowing researchers to instantly visualize trends, spot outliers, and compare the health status of a control group against stressed experimental batches side-by-side.

	Control Plant	Inoculated Plant
Health Indices		
NDVI (800/670)	0.58231	0.42452
GNDVI	0.36079	0.24967
RVI	2.90605	2.12684
WI	1.04077	1.03530
NDWI	0.01967	0.01649
SIPI	0.94147	1.07841
PRI	-0.02723	0.00402
ARI	1.08891	1.21597
CARI	3.53524	2.08058

Chart generated to compare indices between samples

The pipeline automatically calculates eight fundamental biometric indices. **NDVI** and **GNDVI** evaluate overall structural vigor and chlorophyll content, while **SAVI** refines these measurements by suppressing background (soil/pot) interference. For early stress detection, **PRI** measures photosynthetic light-use efficiency, and **ARI** and **CRI** quantify anthocyanins and carotenoids respectively, acting as markers for oxidative stress and senescence. On the thermal side, **CWSI** maps water stress by detecting stomatal closure. Finally, specific pathological indices, like **YRI** (Yellow Rust Index), isolate spectral signatures unique to fungal infections even before visible necrosis appears. These calculated indices are directly linked to specific biological phenomena, a relationship detailed for users via the embedded `explain_spectral_index()` module.

By combining these extracted statistics and visualizations, researchers can get agronomic insights. Furthermore, it allows researchers to input custom algorithmic formulas directly into the system to generate new heatmaps on the fly and retrieve absolute values using `calculate_mean_custom_index()`. By analyzing these specific spectral patterns, the CROPSENSE system can detect the early onset of diseases like Wheat Yellow Rust or Septoria days or even weeks before any necrotic spots or discoloration are visible to the naked eye with applications to disease detection and better understanding of symptoms of diseases.

2.4. Web Application

2.4.1. Introduction

The CropSense project aims to develop a platform capable of processing and analysing plant images, the objective being the transformation of complex datasets into interpretable results that can support decision making in agricultural environments. The project's code can be found accessing the following [link](#).

As part of this initiative, a web application was designed and implemented to serve both a visualization interface and an interaction layer for the underlying data processing pipeline. The users can access project documentation, explore research results, and execute the ETL pipeline. Both intermediate and final results can be visualized in a structured and user-friendly environment.

The system incorporates multiple stages of data processing, for example hyperspectral preprocessing, segmentation and reflectance conversion, and feature extraction. Users are able to interact with these stages by adjusting the parameters such as preprocessing methods, thresholds, and select spectral wavelengths. The results of the analysis are represented as images, spectral graphs, and histograms.

2.4.2. Technologies used

CropSense web application is developed with a modern, modular tech stack that ensures scalability and efficiency when it comes to scientific data processing.

The backend uses Flask, which is a lightweight Python web framework that supports rapid development and a clean separation of concerns. Data processing is managed in Python by NumPy and Pandas libraries that are used for numerical computation and data manipulation.

Matplotlib library is used for visualization of the results, as they are displayed in the web interface. For image handling and file formats, the application uses Tiff file, which supports the processing of multispectral and thermal images used in the analysis pipeline.

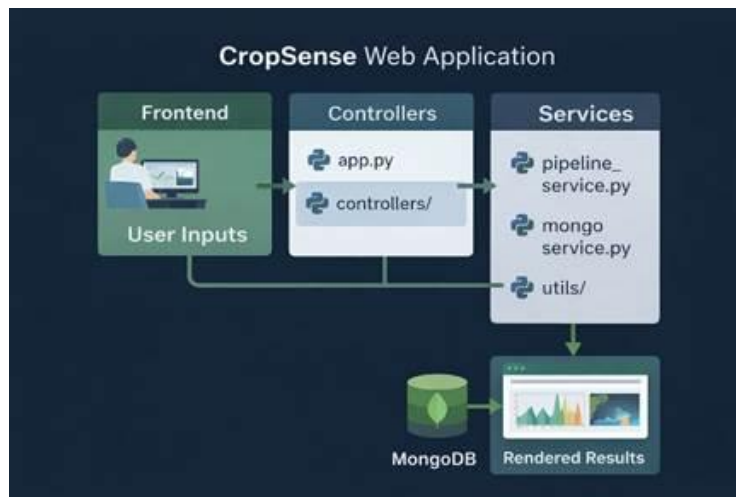
The frontend is developed using standard web technologies (HTML, CSS, and JavaScript), the design being managed with Bootstrap. This library ensures a responsive design that is consistent and user-friendly across different devices.

For data storage, the application uses MongoDB, which is a NoSQL database well-suited for handling large amounts of unstructured data, such as images.

Docker is used to package the application as a container image, which simplifies deployment and improves compatibility when it comes to different work environments by ensuring that the same dependencies and configuration are preserved.

2.4.3. Architecture of the application

The CropSense web application has a modular, layered architecture that ensures clear separation of concerns across the user interface, application logic, data processing, and storage components, thereby simplifying maintainability, scalability, and code readability. This design follows established principles of software architecture commonly used in web applications.



Architecture of CropSense Web Application

2.4.3.1. Presentation Layer (Frontend)

The presentation layer utilizes HTML templates stored in the **templates/** directory, styled with CSS from **static/css/**, and enhanced with JavaScript for dynamic interactions. This layer handles user interface display, parameter selection, and rendering dynamic results. Static assets

reside in **static/** with subdirectories **assets/images/** for predefined visuals and **generated/** for pipeline outputs.

2.4.3.2. Application Layer (Controllers)

Controllers in the **controllers/** directory, particularly `pipeline_controller.py`, manage HTTP requests from the frontend, extract form inputs, invoke service functions, and return rendered templates with processed results. The main application entry point `app.py` defines routes mapped to these controllers.

2.4.3.3. Service Layer (Business Logic)

The service layer in **services/** encapsulates core business logic independent of the web framework. `pipeline_service.py` implements the ETL pipeline, while `mongo_service.py` handles database interactions.

2.4.3.4. Data Storage Layer

MongoDB integration facilitates storage of processed data, metadata, and images from multispectral/thermal sensors, leveraging its NoSQL capabilities for heterogeneous, large-volume data.

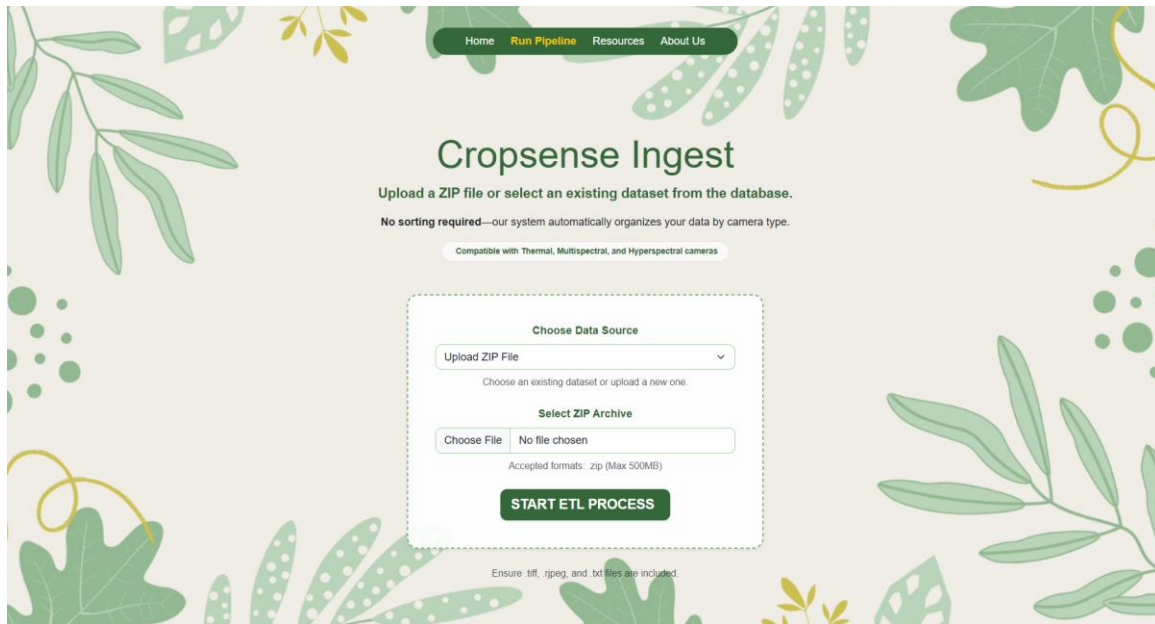
2.4.3.5. Deployment Layer

Containerization via Dockerfile, dependency management through `requirements.txt`, and local development with `.venv` ensure consistent deployment across environments.

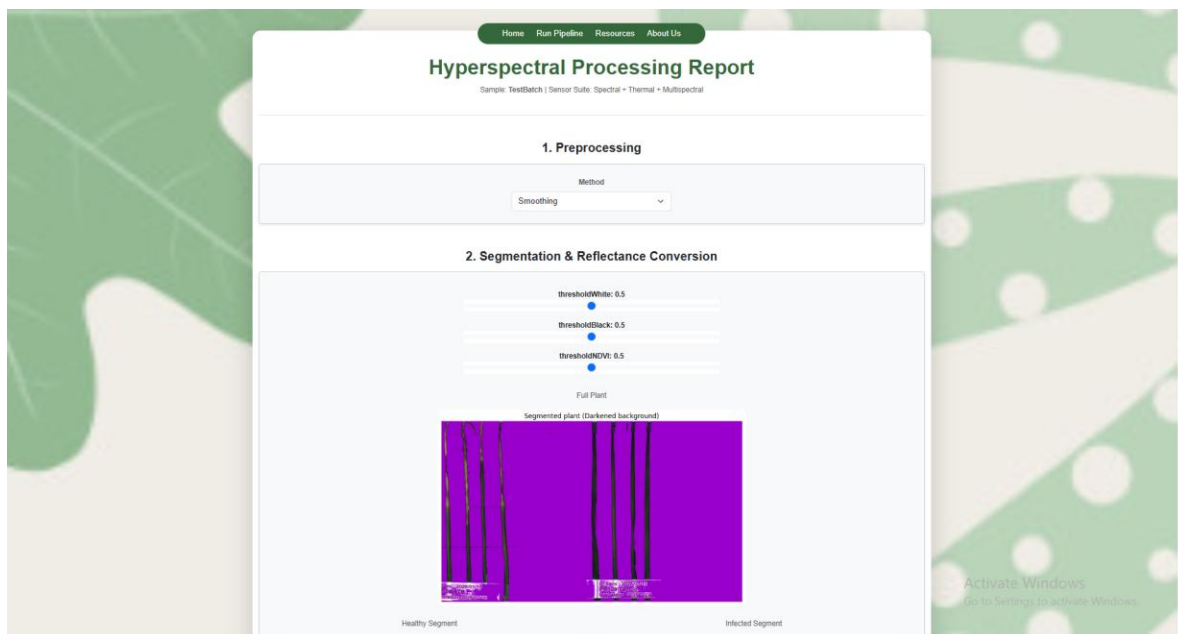
2.4.4. Application Pages

The application consists of four principal pages, interconnected through a navigation bar for seamless user navigation. The Home page provides a comprehensive project overview, while the Resources page centralizes all documentation, final presentation materials, source code, and technical documentation. The About Us page presents detailed information about the development team.

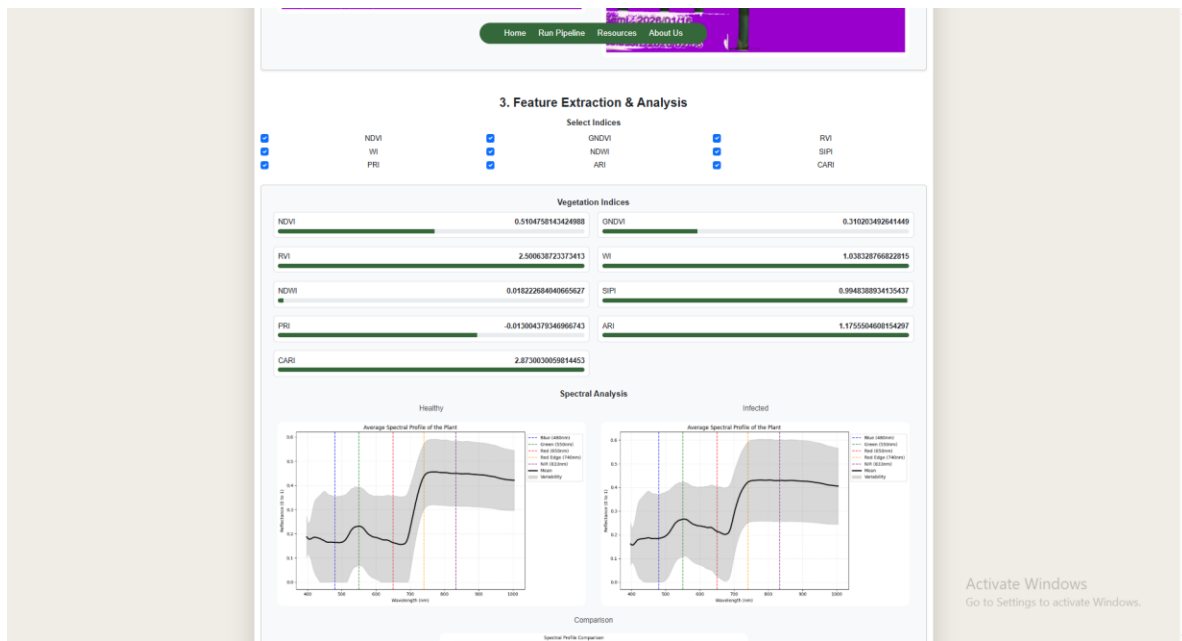
The Pipeline Execution page constitutes the application's core functionality. Users can either upload a ZIP archive containing the requisite analysis files or select from pre-existing datasets stored in the database. Following data selection, the ETL pipeline can be executed, generating an interactive dashboard that displays processed results, vegetation indices, and annotated images. This dashboard supports real-time parameter adjustment, enabling users to iteratively refine analyses or initiate new pipeline executions without page navigation.



Upload ZIP Archive/Select from Database Page



Part of the Result Page



Part of the Result Page

3. System Integration

This section presents how all components of the system are integrated into a complete end-to-end workflow, spanning from image acquisition to data processing and final visualization.

The process begins with three imaging devices: a multispectral camera (Lumo Scanner), a hyperspectral camera (Color Shades Lab), and a thermal camera (Workswell CorePlayer). These devices are used to capture complementary types of imagery, each providing different spectral or thermal information relevant for plant analysis. The images collected from these cameras are then directed into a unified image acquisition stage, where the raw data is organized for further processing.

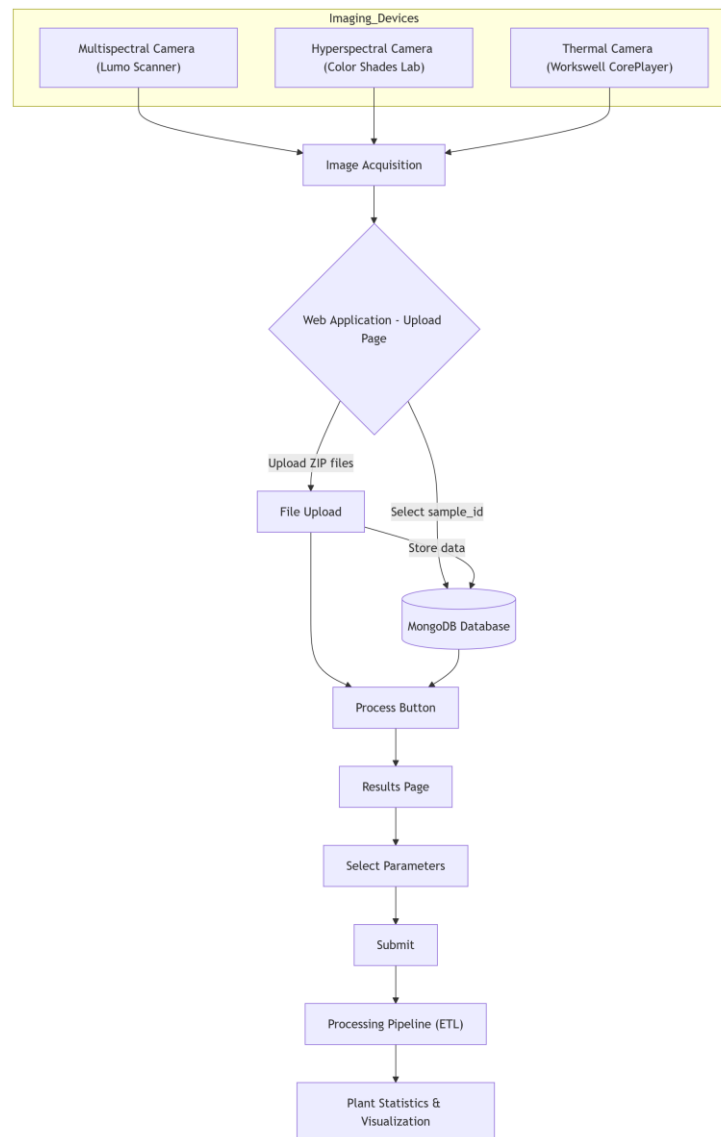
Once acquired, the data is handled through a web application that serves as the central user interface. On the upload page, users have two options: either upload a ZIP file containing all images captured by the different cameras or select an existing sample identifier (sample_id) stored in a MongoDB database. If a ZIP file is uploaded, it is stored directly in the database to ensure consistency and future accessibility of the dataset.

After data selection or upload, the user triggers the processing phase by clicking the process button. This action redirects the workflow to the results page, where users can define specific parameters relevant to the analysis. Once these parameters are selected and submitted, the system initiates the processing pipeline.

The pipeline performs an ETL process, where the raw imaging data is extracted from storage, transformed through computational analysis steps, and structured into meaningful outputs.

Finally, the processed results are presented back in the web interface as plant statistics and visualizations, enabling users to interpret the health and condition of the plant based on the multimodal imaging data.

The pipeline ensures the scientific validity of the project by linking the sensor data directly to the experimental log. To adapt to the sometimes unstable network connectivity found in research environments, as well as being able to reconnect to a remote server, the architecture implements a resilient database connection strategy. The system is configured to test multiple IP addresses, allowing for an automatic fallback from a centralized remote server to a local host (`localhost`). This built-in redundancy guarantees that data ingestion and analysis logging are never interrupted by a network failure, effectively securing the entire workflow from initial acquisition to final storage.



4. Project Management

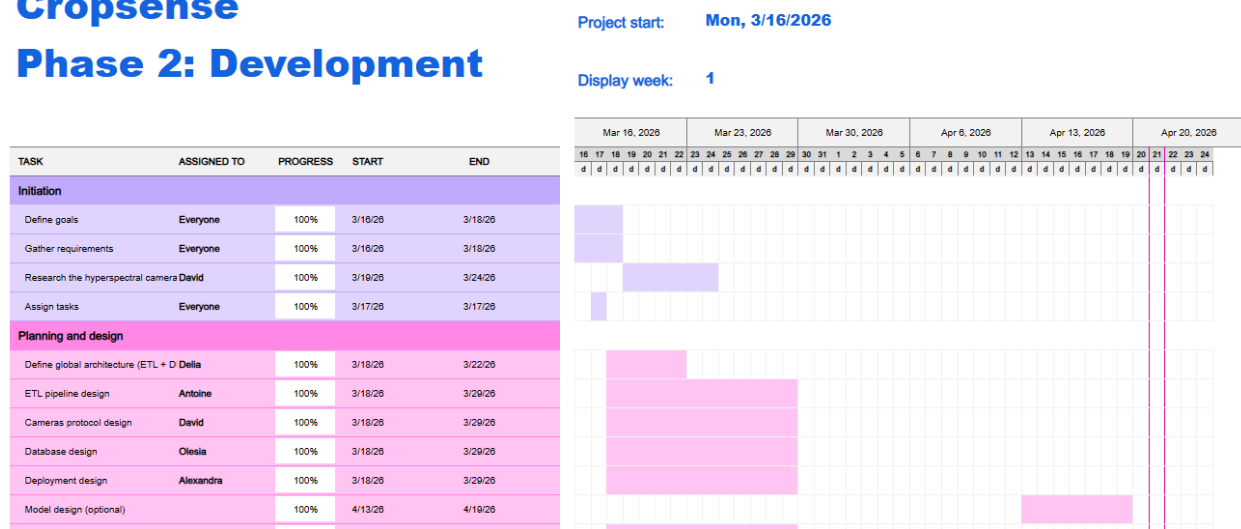
The project was organized into three distinct workstreams: website development, database architecture and ETL pipeline implementation, and camera usage protocol development.

During the first week, the team was not divided into these workstreams; instead, all members collaboratively focused on planning, defining requirements, and selecting appropriate technologies.

We employed a Gantt chart to synchronize these tasks and ensure the project remained on schedule:

Cropsense

Phase 2: Development



4.1 Group 1: Website Development

Two team members were responsible for the development of the web application.

During the first week (March 23), a review of suitable technologies was conducted, considering the requirement to integrate an existing pipeline developed in Python. As a result, a compatible framework, Flask, was selected.

During the second and third weeks (March 30 and April 6), the frontend of the application was designed and implemented, including all main pages such as the resources section.

In the fourth week (April 13) and the beginning of the fifth week, the processing pipeline was successfully integrated into the web application.

4.2 Group 2: Database Implementation and ETL Pipeline Development

Group members were focused on the development of data infrastructure.

The initial design phase (March 18 – March 29) was dedicated to the ETL pipeline, with first tests on hyperspectral data and database schema design.

Throughout the fourth and fifth weeks (March 30 – April 12), action was moved into the execution phase, refactoring the ETL pipeline with the addition of hyperspectral data handled in the project, fix of previous issues in the code of Phase 1, facilitating code migration to the web application, and completing the database implementation to ensure seamless data flow.

Coordination was maintained through in-person weekly meetings and continuous communication via personal messaging platforms.

4.3 Group 3: Cameras Usage Protocol

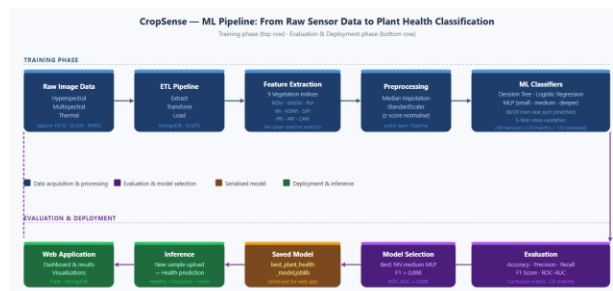
Group 3 focused on the development and application of acquisition protocols for the thermal, multispectral, and hyperspectral cameras.

During the initial phase (March 18 – March 29), work was dedicated to protocol design and research. This included studying the hyperspectral camera and defining acquisition procedures for the thermal and multispectral systems, along with documenting key parameters such as calibration, focus, and acquisition timing.

Throughout the fourth and fifth weeks (March 30 – April 12), research was conducted on alternative display solutions for the thermal camera. In addition, different camera setup configurations and the optimal order for image acquisition were studied and documented.

5. Future Perspectives

The CropSense ML module was designed to train three classifiers — a Decision Tree, Logistic Regression, and a Multilayer Perceptron (MLP) neural network — on vegetation indices extracted from hyperspectral and multispectral acquisitions. The nine input features are: NDVI, GNDVI, RVI, WI, NDWI, SIPI, PRI, ARI, and CARI. The binary classification task distinguishes between healthy (label = 1) and diseased (label = 0) plants.

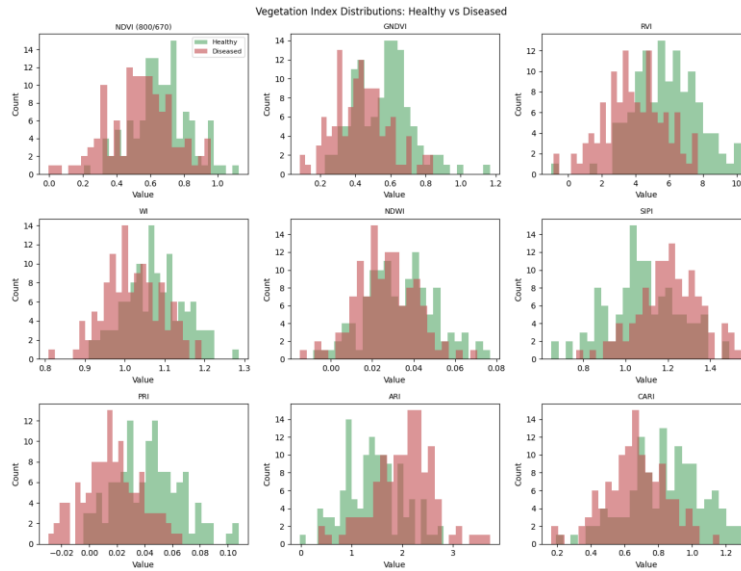


End-to-end CropSense ML pipeline: training phase (left to right) and deployment phase (right to left).

The pipeline proceeds through five stages:

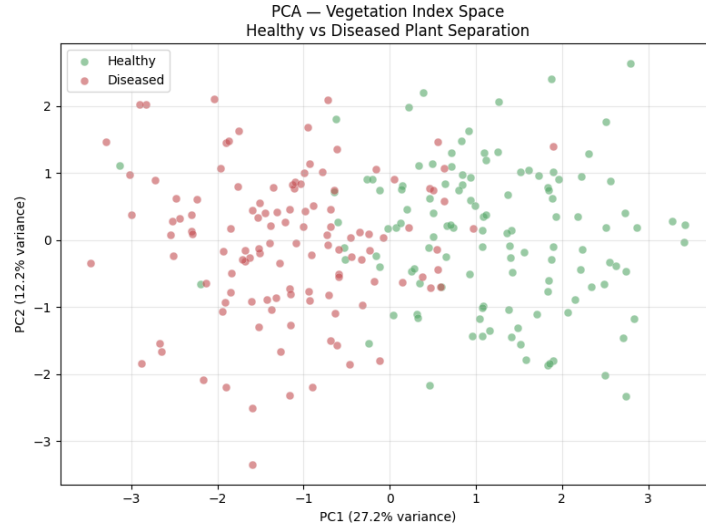
- Raw image data from three sensor modalities (hyperspectral, multispectral, thermal) is collected and stored via the ETL pipeline in MongoDB / GridFS.
- Nine vegetation indices are computed per sample as input features: NDVI, GNDVI, RVI, WI, NDWI, SIPI, PRI, ARI, and CARI.
- A preprocessing step applies median imputation for missing values and z-score normalisation (StandardScaler) before model training.
- Three classifier families are trained: Decision Tree, Logistic Regression, and three MLP variants (small, medium, deeper).
- The best-performing model is serialised to `best_plant_health_model.joblib` and made available for inference on new samples via the web application.

As real labelled ground-truth data from the CropSense acquisition sessions was not yet available at the time of writing, a synthetic dataset was generated to validate the pipeline architecture. The synthetic dataset contains 240 samples (120 healthy, 120 diseased). Each sample is described by nine vegetation indices. The distributions of these indices across classes are shown below.



Vegetation index distributions for healthy (green) and diseased (red) plants. NDVI, GNDVI, and RVI shows the clearest class separation.

A Principal Component Analysis (PCA) was also applied to the standardised feature space to visualise class separability in two dimensions.



PCA projection of the 9-dimensional vegetation index space. The two classes show partial but meaningful separation, confirming that the features carry discriminating information.

Five classifiers were evaluated within a scikit-learn Pipeline:

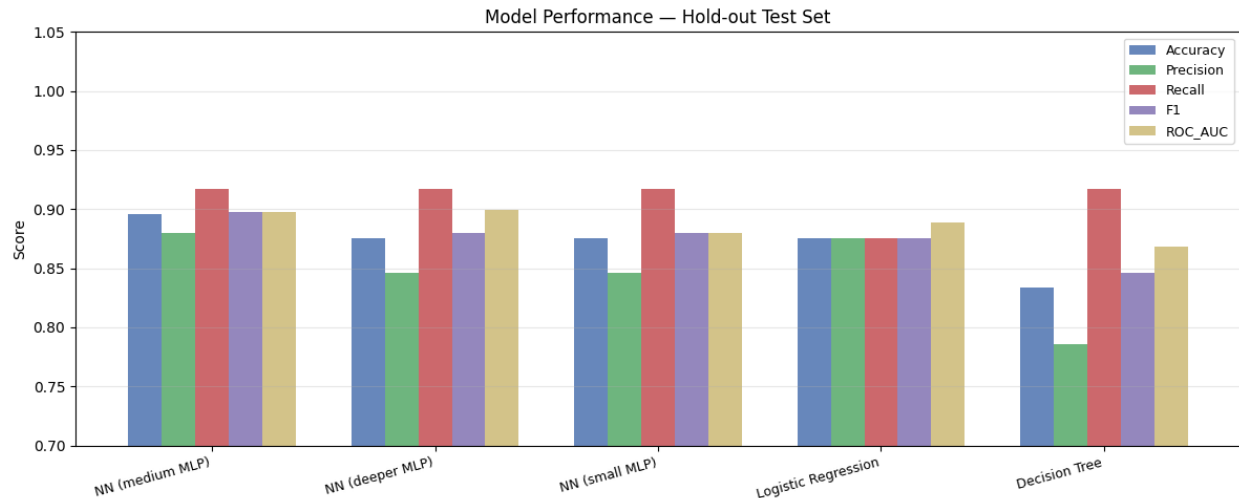
- Decision Tree (max_depth = 5, min_samples_leaf = 5)
- Logistic Regression (balanced class weights, max_iter = 2 000)
- NN small MLP — 1 hidden layer of 16 units
- NN medium MLP — 2 hidden layers of 32 and 16 units
- NN deeper MLP — 3 hidden layers of 64, 32, and 16 units

Each pipeline applies median imputation followed by StandardScaler normalisation before the classifier. The dataset was split 80/20 (stratified) for hold-out evaluation, and a 5-fold stratified cross-validation was performed to estimate generalisation stability.

Table below summarizes performance on the 20% hold-out test set. Bold rows indicate the best-performing model.

Table 5. Prediction models performances

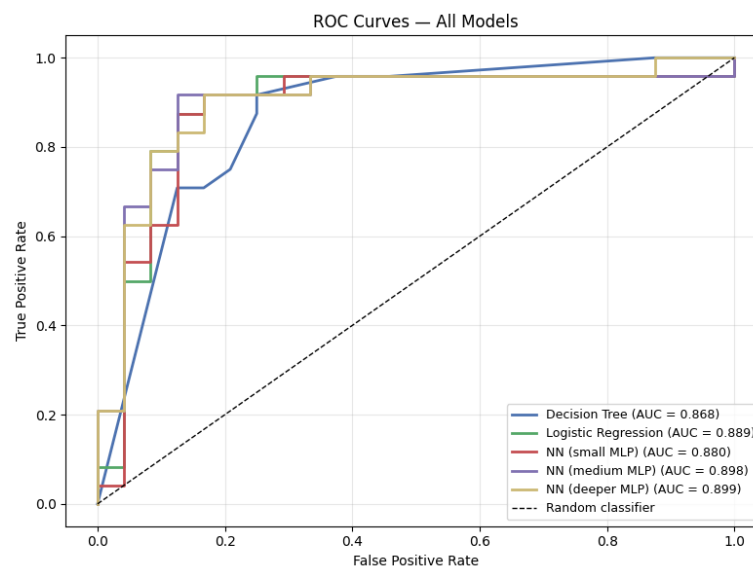
Model	Accuracy	Precision	Recall	F1	ROC-AUC
NN (medium MLP)	0.8958	0.8800	0.9167	0.8980	0.8976
NN (deeper MLP)	0.8750	0.8462	0.9167	0.8800	0.8993
NN (small MLP)	0.8750	0.8462	0.9167	0.8800	0.8802
Logistic Regression	0.8750	0.8750	0.8750	0.8750	0.8889
Decision Tree	0.8333	0.7857	0.9167	0.8462	0.8681



Model performance comparison across all five metrics on the hold-out test set

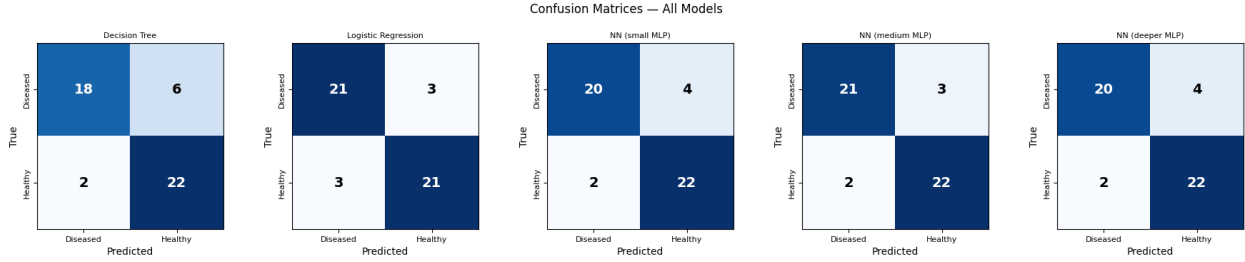
The NN (medium MLP) achieved the highest overall performance (Accuracy = 0.896, F1 = 0.898, ROC-AUC = 0.898). NDVI and RVI were the dominant discriminating features in the Decision Tree, consistent with their known sensitivity to chlorophyll content reduction in diseased tissue.

The ROC curves confirm that all models perform well above the random baseline, with the MLP variants and Logistic Regression reaching AUC values above 0.88.



ROC curves for all classifiers. AUC values above 0.88 confirm strong discriminating power across all models.

The confusion matrices below reveal the error patterns of each model on the test set.

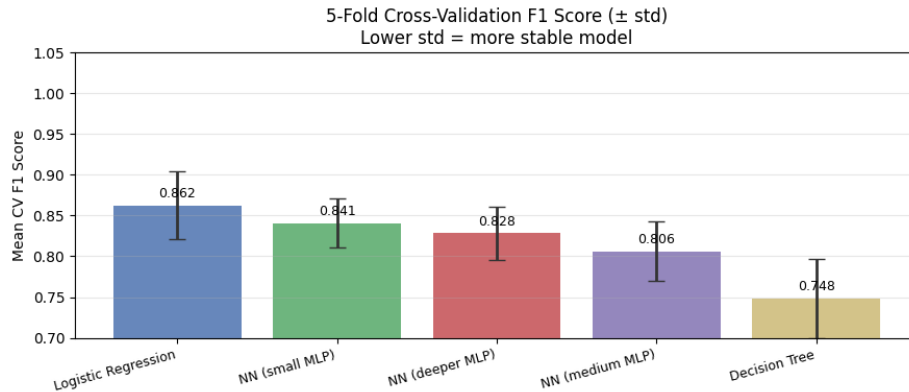


Confusion matrices (test set). All MLP variants show low false-negative rates, a critical property for disease detection.

Hold-out performance alone can be misleading on small datasets. The 5-fold cross-validation results below provide a more robust estimate of generalisation.

Table 5. 5-fold cross-validation results (mean $\pm$ std). Lower std indicates more stable generalisation.

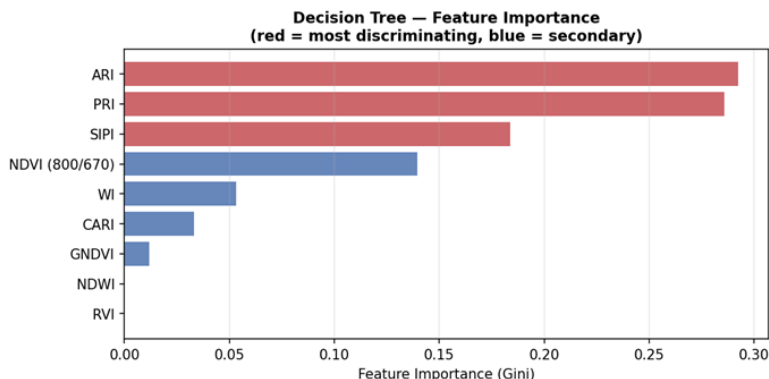
Model	CV F1 (mean)	CV F1 (std)	CV ROC-AUC
Logistic Regression	0.8624	0.0416	0.9278
NN (small MLP)	0.8407	0.0305	0.9125
NN (deeper MLP)	0.8282	0.0331	0.8698
NN (medium MLP)	0.8062	0.0360	0.8691
Decision Tree	0.7483	0.0485	0.7790



Mean CV F1 with standard deviation. Logistic Regression achieves the best CV F1 (0.862), highlighting that simpler models can generalise well on limited data.

An important nuance emerges from the cross-validation: while the medium MLP achieves the best hold-out F1 (0.898), Logistic Regression achieves the best CV F1 (0.862) with the smallest std (0.042), suggesting it may generalise more reliably on the current small dataset. With a larger real

dataset, the deeper MLP architectures are expected to outperform Logistic Regression, as they can capture non-linear spectral interactions that a linear model cannot represent.



Decision Tree feature importances. NDVI and RVI (red) are the most discriminating indices, aligning with published findings on chlorophyll-related stress markers.

Several steps remain before the module can operate on real experimental data:

1. Produce `plant_features.csv` linking each `sample_id` to a confirmed health label from actual acquisition sessions. The ML module reads this file directly.
2. Implement the Workswell TIFF reader to populate the three thermal metrics (`cwsi`, `max_leaf_temp_c`, `ambient_temp_c`), which are currently stored as `None`. Adding these features is expected to improve recall for early-stage stress detection.
3. Retrain the pipeline on real data once labelled samples are available; the deeper MLP architectures are likely to outperform simpler models at larger scales.
4. Integrate model inference into the web application: upload a new sample and receive a real-time health prediction (Healthy / Diseased + confidence score) via the serialised `best_plant_health_model.joblib` file.
5. Consider adding explainability outputs (e.g. SHAP values) so agronomists can understand which spectral bands contributed most to each prediction.

Despite being trained on synthetic data, these results demonstrate that the pipeline architecture is sound and that vegetation indices carry sufficient discriminating power for binary health classification. The methodology is therefore ready to be applied directly to real CropSense acquisitions as soon as labelled data becomes available.

Web Application:

Future work on the web application will focus on improving both deployment and usability aspects.

A Docker image has already been created, enabling streamlined deployment and facilitating scalability across different environments.

In addition, a key objective is the development of an user-friendly interface that allows users to upload plant images and receive predictions on whether the plant is affected by disease.

6. Conclusion

The CropSense Phase 2 project successfully achieved its primary objective of designing and integrating a complete imaging and data processing platform for plant analysis. The system combines multispectral, hyperspectral, and thermal imaging with a robust ETL pipeline, a scalable database architecture, and an interactive web application. Together, these components form a coherent end-to-end workflow, from data acquisition to visualization and preliminary agronomic interpretation.

The results demonstrate that the platform is capable of reliably acquiring, structuring, and processing heterogeneous sensor data. The implementation of standardized acquisition protocols ensured data consistency, while the automated pipeline enabled efficient ingestion, transformation, and visualization of complex datasets. The generated outputs, such as vegetation indices, heatmaps, and spectral signatures, confirm that the system can extract meaningful physiological information from plants and make it accessible to users.

In conclusion, the CropSense system provides a solid technical and methodological foundation for precision agriculture applications. While some hypotheses require further validation with real datasets, the current results demonstrate the feasibility and relevance of the approach. Future work will focus on integrating real labeled data, improving model robustness, and enhancing the platform's usability for practical deployment.

7. References

- Flanagan, D., 2011. *JavaScript: The definitive guide*, 6th ed. O'Reilly Media.
- Fowler, M., 2002. *Patterns of enterprise application architecture*. Addison-Wesley.
- Hunter, J.D., 2007. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering* 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Mahlein, A.-K., 2016. Plant disease detection by imaging sensors – Parallels and specific demands for precision agriculture and plant phenotyping. *Plant Disease* 100, 241–251. <https://doi.org/10.1094/PDIS-03-15-0340-FE>
- McKinney, W., 2017. *Python for data analysis: Data wrangling with Pandas, NumPy, and IPython*, 2nd ed. O'Reilly Media.
- Merkel, D., 2014. Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal* 2014(239), 2. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>
- Mermaid, n.d. Mermaid Live Editor [Computer software]. <https://mermaid.live/edit>
- MongoDB, Inc., 2024a. MongoDB manual: Data modeling introduction. <https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>
- MongoDB, Inc., 2024b. MongoDB manual: GridFS. <https://www.mongodb.com/docs/manual/core/gridfs/>
- MongoDB, Inc., 2024c. PyMongo documentation. <https://pymongo.readthedocs.io/en/stable/>
- OpenAI, 2026. ChatGPT (Apr 18 version) [Large language model]. <https://chatgpt.com/>
- Pallets Projects, n.d. Flask documentation. <https://flask.palletsprojects.com/en/stable/>
- Seibert, C., 2023. tiff file: Read and write TIFF files [Python library]. <https://pypi.org/project/tiff file/>